Design

From System Specification to Structure

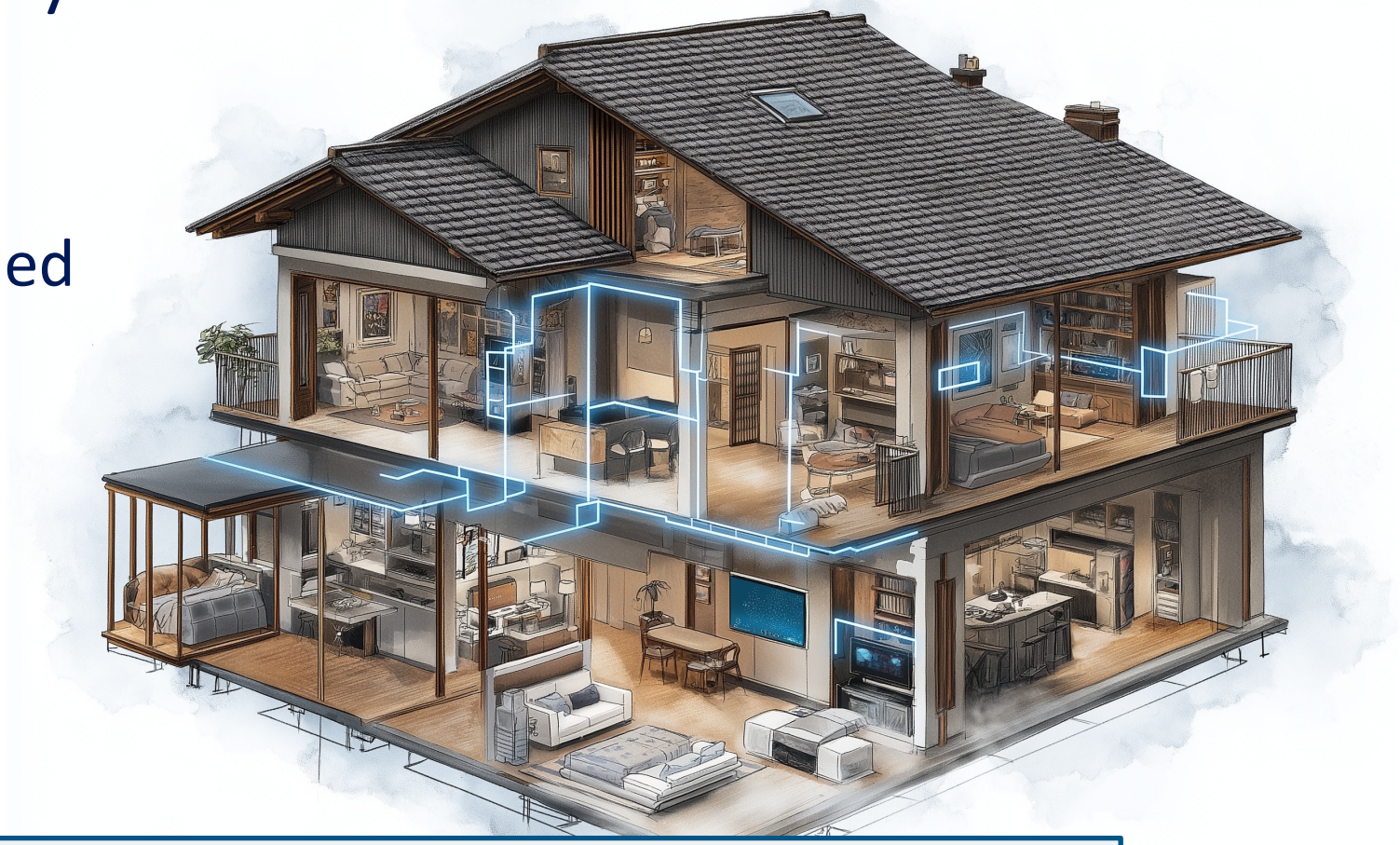
FINTECH 512
Software Engineering

Duke

Design: How the System Is Structured

Design determines how:

- responsibilities are assigned
- components interact
- changes are isolated



Design choices determine whether a system can **keep its promises over time.**

Duke

What Design Fights Against

Design exists because systems face:

- Growing complexity
- Changing requirements
- Multiple teams and viewpoints
- Uncertainty about the environment

Good design preserves system guarantees despite these forces.

Duke

Design Under Constraints

- Design problems are **wicked**: requirements evolve and interact
- Design is **iterative and emergent**, not linear
- Every design involves **tradeoffs** among competing forces
- Designs are judged by how well they satisfy **explicit goals**

Good design makes tradeoffs *intentionally* — not accidentally.

Design Qualities That Preserve Guarantees

Good designs tend to have:

- **Minimal complexity**: behavior is easier to reason about
- **Ease of maintenance**: changes don't break existing guarantees
- **Low coupling, high cohesion**: changes stay localized
- **Extensibility**: new behavior can be added safely
- **Reusability**: behavior is well-defined and isolated
- **Portability**: guarantees survive environmental change

Coupling and Cohesion

Cohesion
how well a module's responsibilities form a single, focused purpose

Coupling
how much a module must know about other modules to function

Good design maximizes cohesion and minimizes coupling to contain the impact of change.

1. Yourdon, Edward; Constantine, Larry LeRoy (1979) [1975]. Structured Design: Fundamentals of a Discipline of Computer Program and Systems Design
2. H. Washizaki, eds., Guide to the Software Engineering Body of Knowledge (SWEBOK Guide), Version 4.0, IEEE Computer Society, 2024; www.swebok.org

Duke

Cohesion at the Routine Level

Cohesion describes whether a routine:

- has **one clear responsibility**
- supports **one system-level behavior**
- is easy to **reason about and test**

High cohesion

- routine does one thing
- all statements contribute to that purpose

```python
def calculate_sum(numbers):
    total = 0
    for num in numbers:
        total += num
    return total
```

Low cohesion

- routine mixes unrelated responsibilities
- behavior becomes harder to specify and verify

Low cohesion hides specification decisions inside code.

```python
def process_and_log_data(data):
    total = sum(data)
    print(f"Sum: {total}")
    # Unrelated task
    with open("output.txt", "w") as file:
        file.write("Data processed successfully.")
```

Duke

Cohesion is a Spectrum

High

Functional	Routine performs one and only one operation	
Sequential	Steps in the routine must occur in a specific sequence, where output of one step is input to the next.	A routine that reads data, processes it, and then writes the result.
Communicational	Operations in the routine work on the same data or contribute to the same goal.	A routine that calculates the average, min, and max of a single data array.
Temporal	Actions in the routine are grouped because they occur at the same time in execution.	A routine that initializes multiple unrelated system components during startup.
Procedural	Actions in the routine are grouped because they must follow a specific control sequence.	A routine that performs unrelated steps in a specific order, such as input validation followed by logging.
Logical	Actions are grouped because they perform similar tasks, but the specific task is chosen by a control variable.	A routine that handles both error logging and user notifications based on a flag.
Coincidental	Actions have no meaningful relationship and are grouped arbitrarily.	A routine that logs messages, calculates sums, and clears memory, all in one method.

Low

Duke

Temporal

```
def initialize_system():
    load_config()
    initialize_logging()
    connect_to_database()
    start_background_tasks()
```

Temporal cohesion groups decisions by *when* they happen, not by *what they guarantee*.

Duke

Temporal – Slight factoring

```
def initialize_system():
    init_configuation()
    init_logging()
    init_persistence()
    init_background_processing()
```

- Responsibilities are now named
- Decision are isolated
- Tests become more focus to the responsibility
- Change has a place to go

Duke

Logical

```
def handle_event(event_type, data):
    if event_type == "error":
        log_error(data)
    elif event_type == "warning":
        log_warning(data)
    elif event_type == "info":
        log_info(data)
```

A diagnostic question:

“If we add a new event type tomorrow, where does the change go?”

Duke

Logical Refactoring: Lookup Table

```
HANDLERS = {
    "error": log_error,
    "warning": log_warning,
    "info": log_info,
}

def handle_event(event_type, data):
    try:
        HANDLERS[event_type](data)
    except KeyError:
        raise ValueError(f"Unknown event_type: {event_type}")
```

- New types are added by extending the table, not editing branching logic
- You’re forced to decide what happens on unknown types (spec clarity!)
- Reads like “contract + mapping”

Duke

Logical Refactoring: polymorphism / Strategy

```
class EventHandler:
    def handle(self, data): ...

class ErrorHandler(EventHandler):
    def handle(self, data): log_error(data)

class WarningHandler(EventHandler):
    def handle(self, data): log_warning(data)

class InfoHandler(EventHandler):
    def handle(self, data): log_info(data)

HANDLERS = {
    "error": ErrorHandler(),
    "warning": WarningHandler(),
    "info": InfoHandler(),
}

def handle_event(event_type, data):
    handler = HANDLERS.get(event_type)
    if not handler:
        raise ValueError(f"Unknown event_type: {event_type}")
    handler.handle(data)
```

Duke

High Cohesion: Design Principles

High cohesion means each unit:

- **Owns a single responsibility:** supports one system-level promise
- **Has a clear, meaningful name:** makes intent and guarantees visible
- **Is small enough to reason about:** behavior can be understood and tested
- **Limits side effects:** changes don’t unexpectedly violate other guarantees

High cohesion localizes change and reduces risk.

Coupling: How Change Spreads

High-risk coupling

- One unit depends on another's **internal details**
- Examples:
  - accessing another function's variables
  - shared global state
  - control flags that alter behavior

Medium-risk coupling

- Units share **more information than necessary**
- Example: passing complex objects when only part is needed

Low-risk coupling

- Units interact through **explicit, minimal data**
- Example: passing only required values via parameters

Duke

Coupling Exercise

A

```
def calculate_fee(amount):
    return amount * FEE_RATE
```

B

```
def calculate_fee(transaction):
    return transaction.amount * transaction.fee_rate
```

C

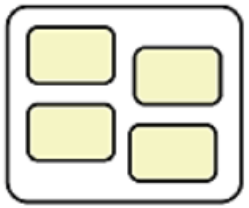
```
def calculate_fee(transaction, mode):
    if mode == "standard":
        return transaction.amount * 0.01
    elif mode == "premium":
        return transaction.amount * 0.005
```

If requirements change tomorrow, which version is most likely to cause bugs — and why?

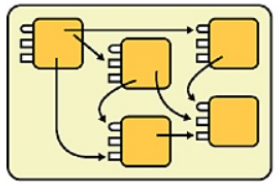
Duke

Design Scales from Systems to Routines

System — what the system guarantees to the outside world



Subsystems / Packages — how responsibilities are divided and isolated



Classes — how related behaviors are grouped

Internal Routine — how individual decisions are expressed

Code Complete, 2nd Ed. Steve McConnell

Duke

Design Principles Apply at Every Level

- Responsibilities define purpose
- Cohesion groups related decisions
- Coupling controls knowledge flow
- Change containment limits risk

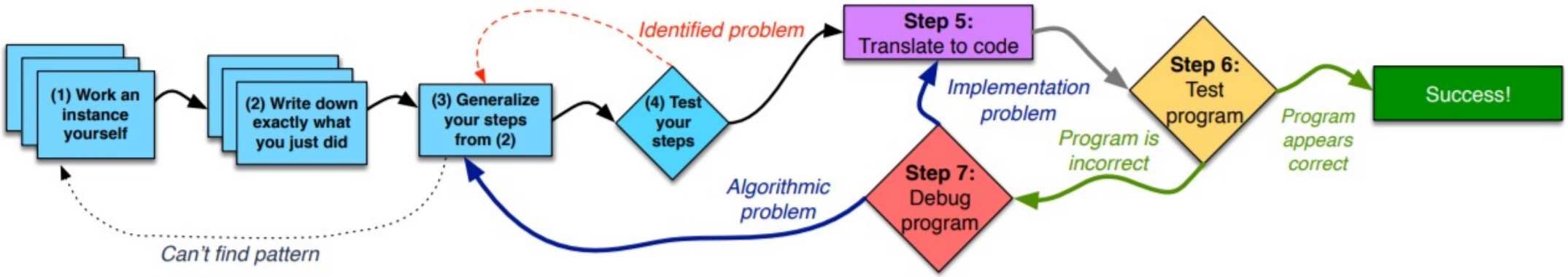
The same ideas, applied repeatedly, create robust systems.

Duke

Levels of Design: Internal Class / Routine

Laying out the detailed functionality of the individual routines

Seven Steps



https://adhilton.pratt.duke.edu/sites/adhilton.pratt.duke.edu/files/u37/seven-steps-comped19.pdf

Duke

Design Details for Routines

When designing a routine, decide explicitly:

- **Purpose**: What single responsibility does this routine own?
- **Inputs**: What data does it require? In what form?
- **Outputs**: What does it return or change?
- **Preconditions**: What must be true before it is called?
- **Postconditions**: What is guaranteed after it completes?
- **Failure behavior**: How does it signal errors or exceptional cases?

Tests as Executable Contracts

Well-designed tests:

- encode **expected behavior**
- make **assumptions explicit**
- validate **postconditions**
- protect guarantees as code evolves

If a behavior matters, it should be testable.
And if it is not testable, it’s not fully designed.

Using Pseudocode to Commit to Design Decisions

Good pseudocode should:

- express **control flow and major decisions**
- reflect **inputs, outputs, and assumptions**
- avoid language-specific syntax
- omit low-level implementation details

If you can’t write pseudocode clearly, the design is not yet clear.

Worked Example: Transaction Fee

We need a routine:

```
fee_cents = calculate_fee(amount_cents)
```

Rules:

- Fee = 1.2% of amount cents
- Minimum fee: 50 cents
- Maximum fee: 500 cents
- Fee is rounded up to the nearest cent

Duke

Worked Example: Tests as Executable Contracts

amount_cents	expected_fee
1000	50
4000	50
50000	500
4167	51

What are some missing test cases?
What concepts have you had previously to find test cases?

Duke

Worked Example: Pseudocode

```
If the amount is not valid, signal an error

Determine the base fee as a percentage of the amount

Round the fee upward to the nearest cent

If the fee is below the minimum, use the minimum fee

If the fee is above the maximum, use the maximum fee

Return the final fee
```

Duke

Design at the Class Level

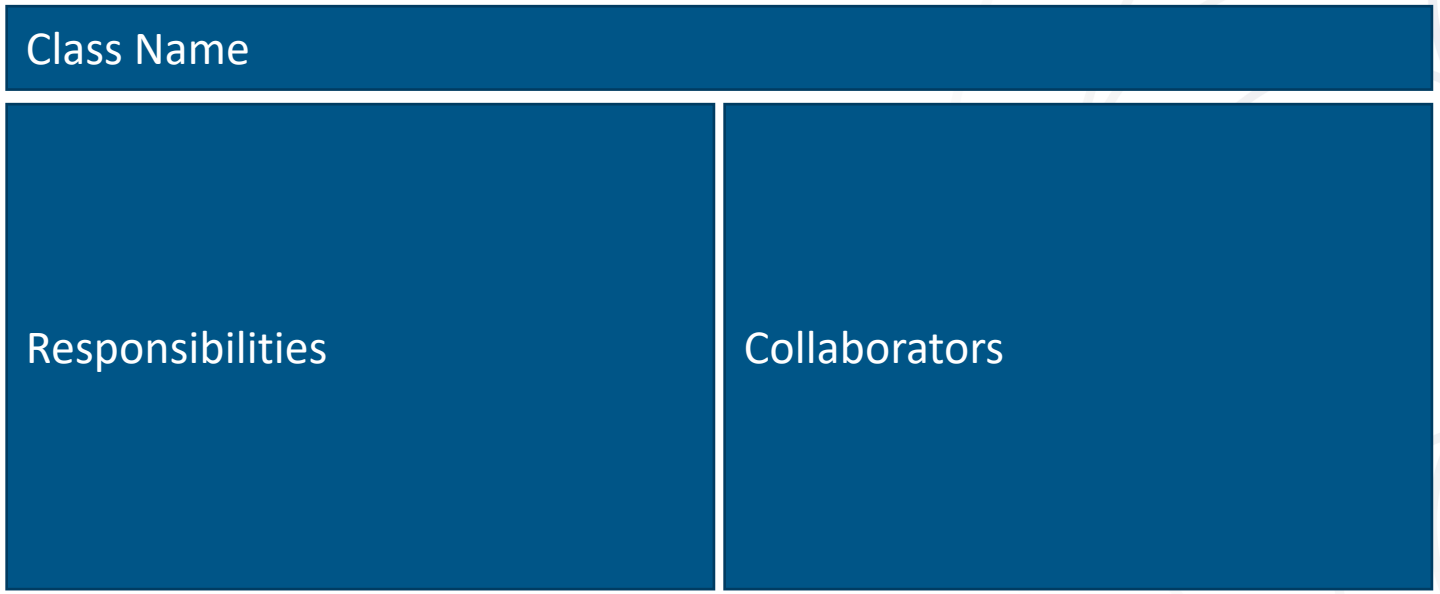
At the class level, design focuses on:

- **Responsibilities:** What related behaviors belong together?
- **Cohesion:** Do all methods support a single purpose?
- **Coupling:** How much does this class need to know about others?
- **Change containment:** Can this responsibility evolve without ripple effects?

Classes group related decisions so change stays local.

CRC Cards: Reasoning About Class Responsibilities

- **Class** — what entity owns this responsibility?
- **Responsibilities** — what behaviors belong together?
- **Collaborators** — who does it need to talk to?



Duke

CRC Card — Fee Calculator

FeeCalculator	
Calculate transaction fees based on defined rules Enforce minimum and maximum fee constraints Apply rounding policy consistently Validate fee-related assumptions (e.g., valid amount)	*(None)* — pure computation *(optionally later: Configuration, PricingPolicy, Logger)*

Duke

Heuristics for Discovering Responsibilities

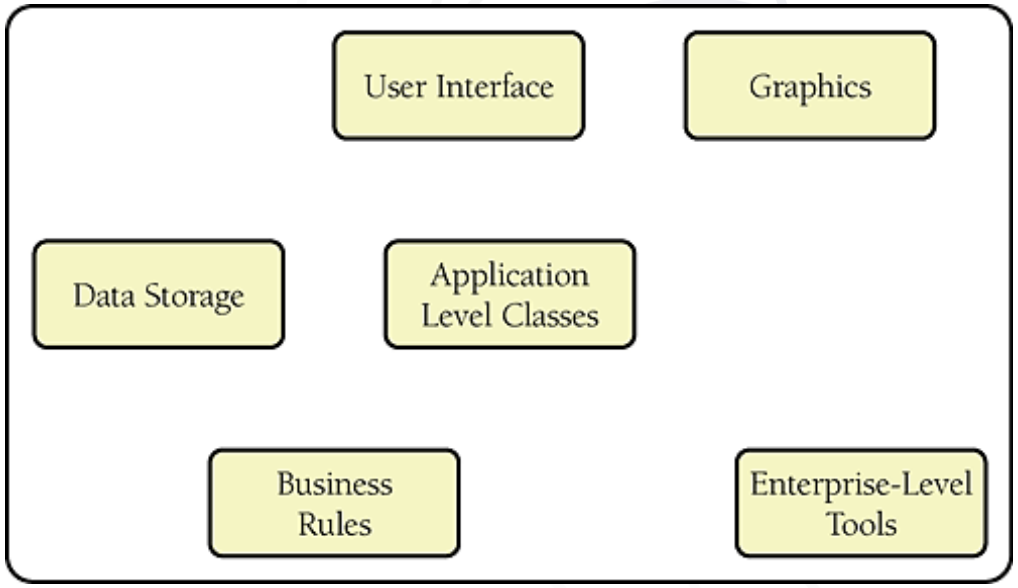
Good candidates for responsibilities often:

- represent a **stable concept** in the problem domain
- own **rules or invariants** that must be enforced
- change for **one primary reason**
- hide **decisions that shouldn't leak**

Design at the Subsystem Level

At the subsystem or package level, design focuses on:

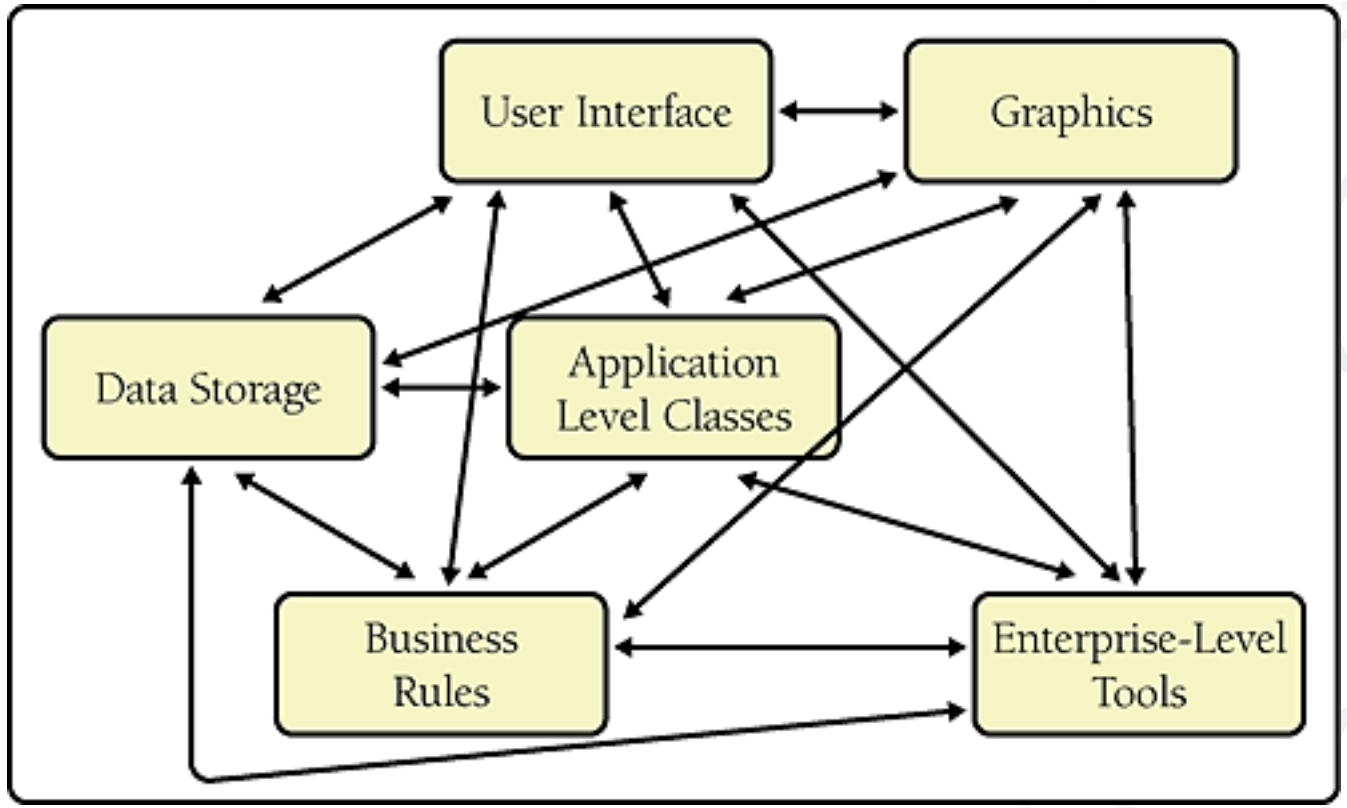
- **Responsibility boundaries:** What part of the system promise does this sub
- **Internal cohesion:** Do the classes inside belong together?
- **External coupling:** How much do other subsystems need to know?
- **Change containment:** Can this subsystem evolve independently?



Code Complete, 2nd Ed. Steve McConnell

Duke

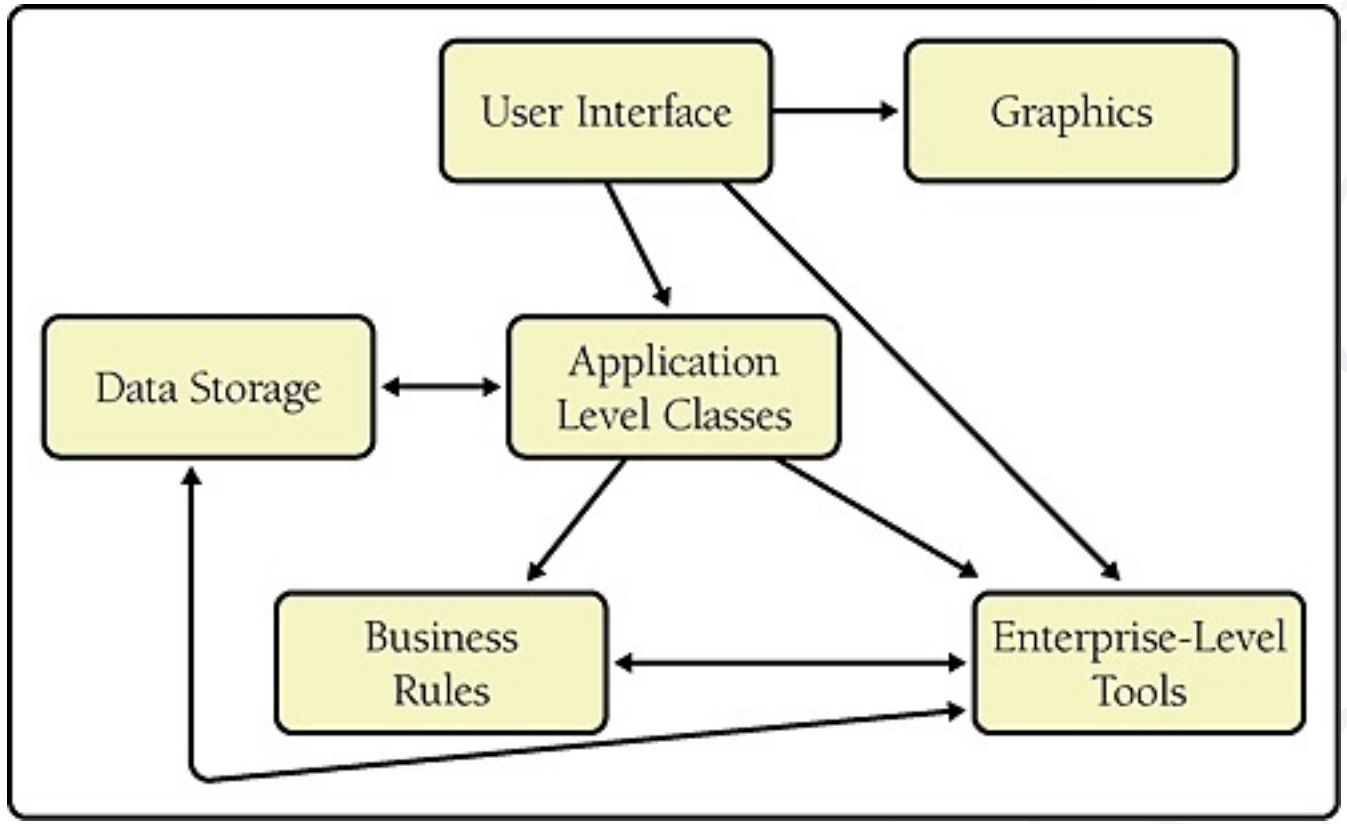
Levels of Design: Subsystems / Packages



Code Complete, 2nd Ed. Steve McConnell

Duke

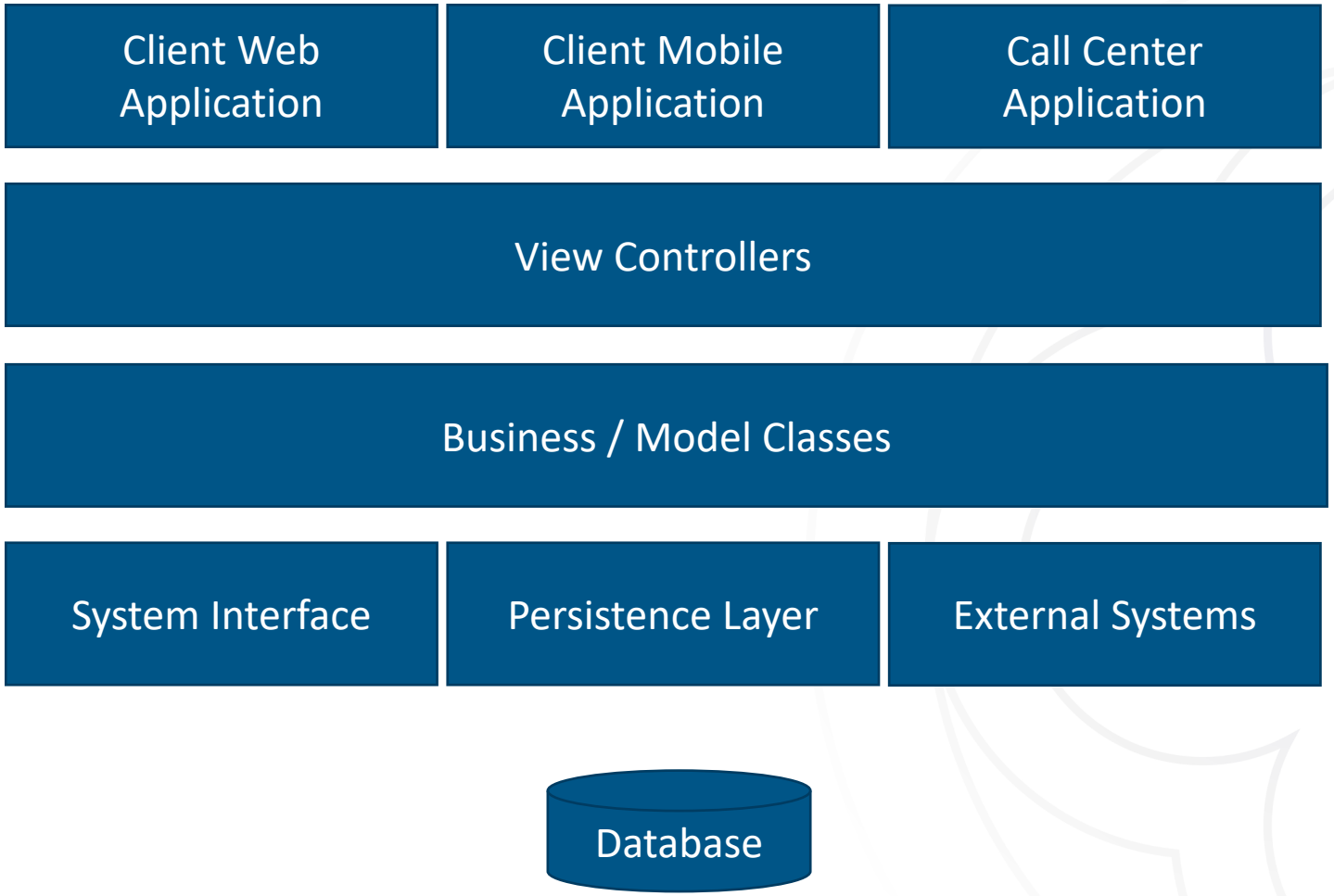
Levels of Design: Subsystems / Packages



Code Complete, 2nd Ed. Steve McConnell

Duke

BluePay: Subsystem / Packages



Duke

Design at the System Level

At the system level, design focuses on:

- **System guarantees**
  What does the system promise to users and external systems?
- **System boundaries**
  What is inside the system — and what is not?
- **Failure modes**
  How does the system behave when things go wrong?
- **Evolution**
  How can the system change without breaking existing guarantees?

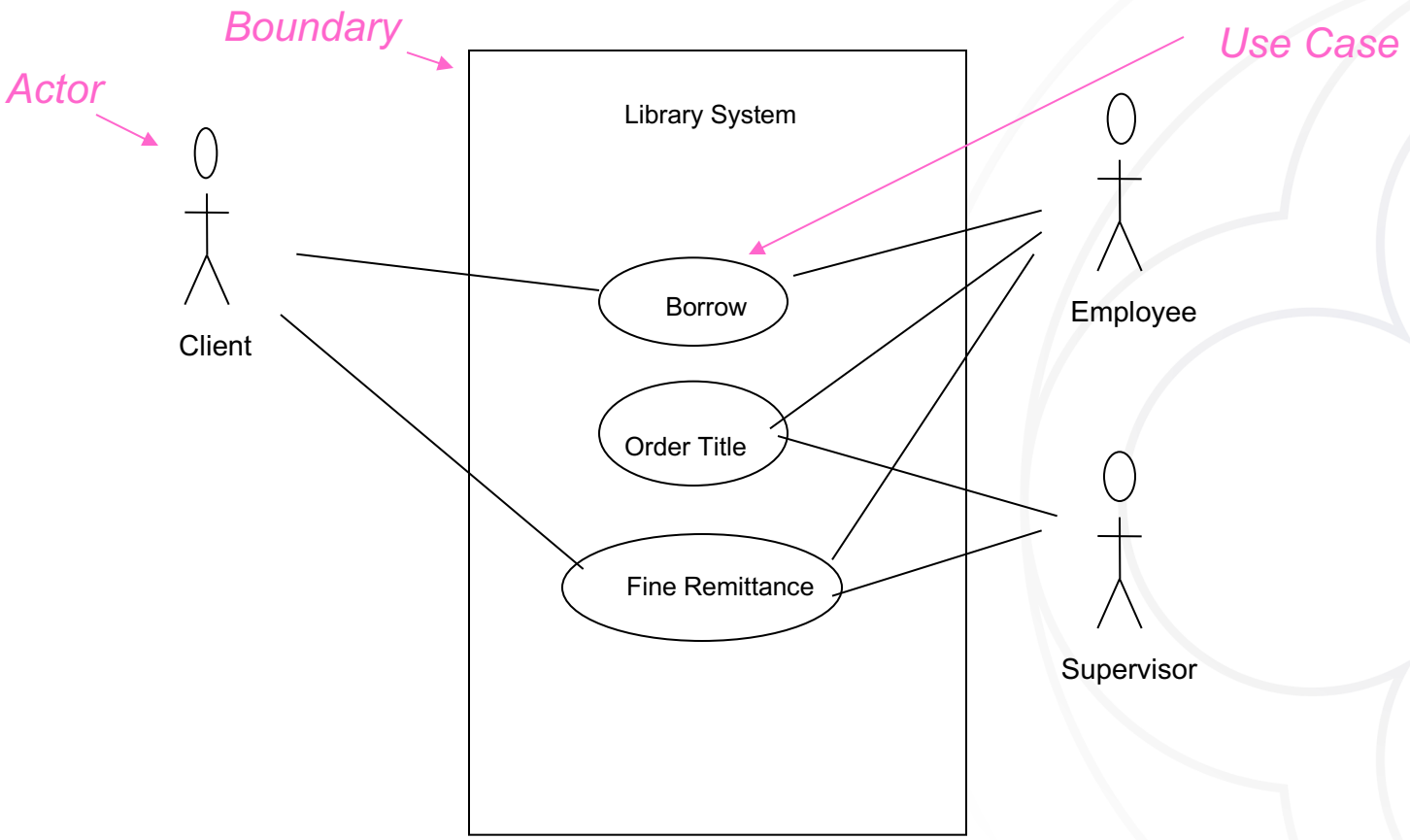
Use cases at the System Level

Use case diagrams help us:

- define **what is inside the system**
- clarify **who interacts with the system**
- identify **system responsibilities**
- avoid scope creep

A use case represents a system promise to an external actor.

Use-Case Diagrams



Duke

Use-Case Diagrams

- **Actors:** A role that a user plays with respect to the system, including human users and other systems. e.g., inanimate physical objects (e.g. robot); an external system that needs some information from the current system.

- **Use case:** A set of scenarios that describing an interaction between a user and a system, including alternatives.

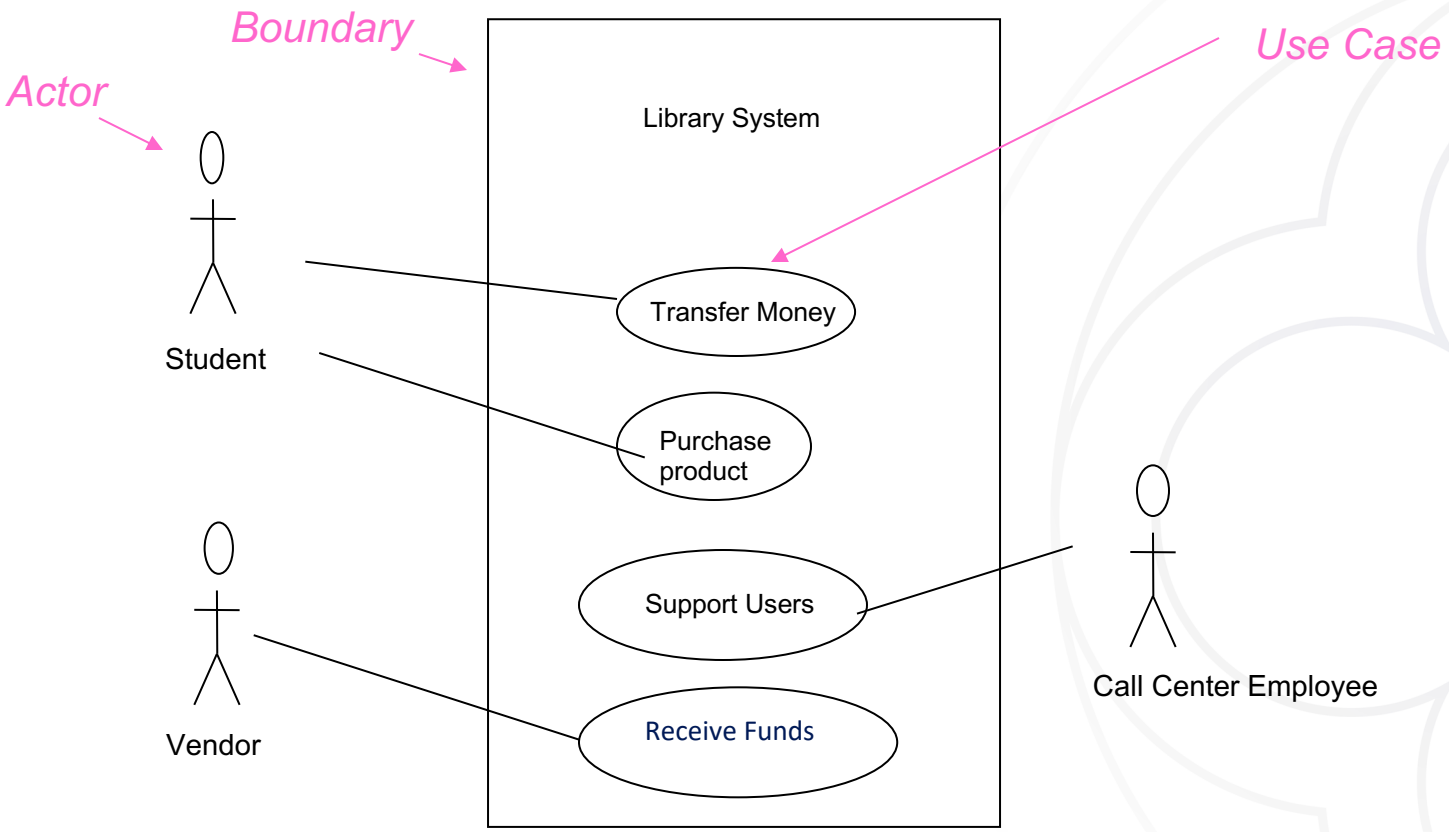
- **System boundary**: rectangle diagram representing the boundary between the actors and the system.

Actor

Use Case

Duke

Blue Pay System



Duke

Using Generative AI in Software Design (Responsibly)

Where Generative AI Helps

- Explore **alternative designs**
- Surface **edge cases and failure modes**
- Draft **tests, specifications, and pseudocode**
- Challenge assumptions (“what did we forget?”)

Where Humans Remain Essential

- Defining **system guarantees**
- Choosing **responsibility boundaries**
- Deciding **what tradeoffs matter**
- Owning **accountability and risk**

AI can generate options — humans must choose.

Duke

Design Is Decision-making at Every Level

- Routines define **local contracts**
- Classes group **cohesive responsibilities**
- Subsystems control **dependency and change**
- Systems define **promises to the outside world**

Good design makes change predictable.